Data Engineering and MLOps in Business

Intro to API & DataBases

Primoz Konda

AAUBS

March 10, 2025

pk@business.aau.dk

Outline

1 Course Approach

2 Introduction

3 API (1)

4 Authentication

5 API (2)

6 Data Bases

7 Exercise

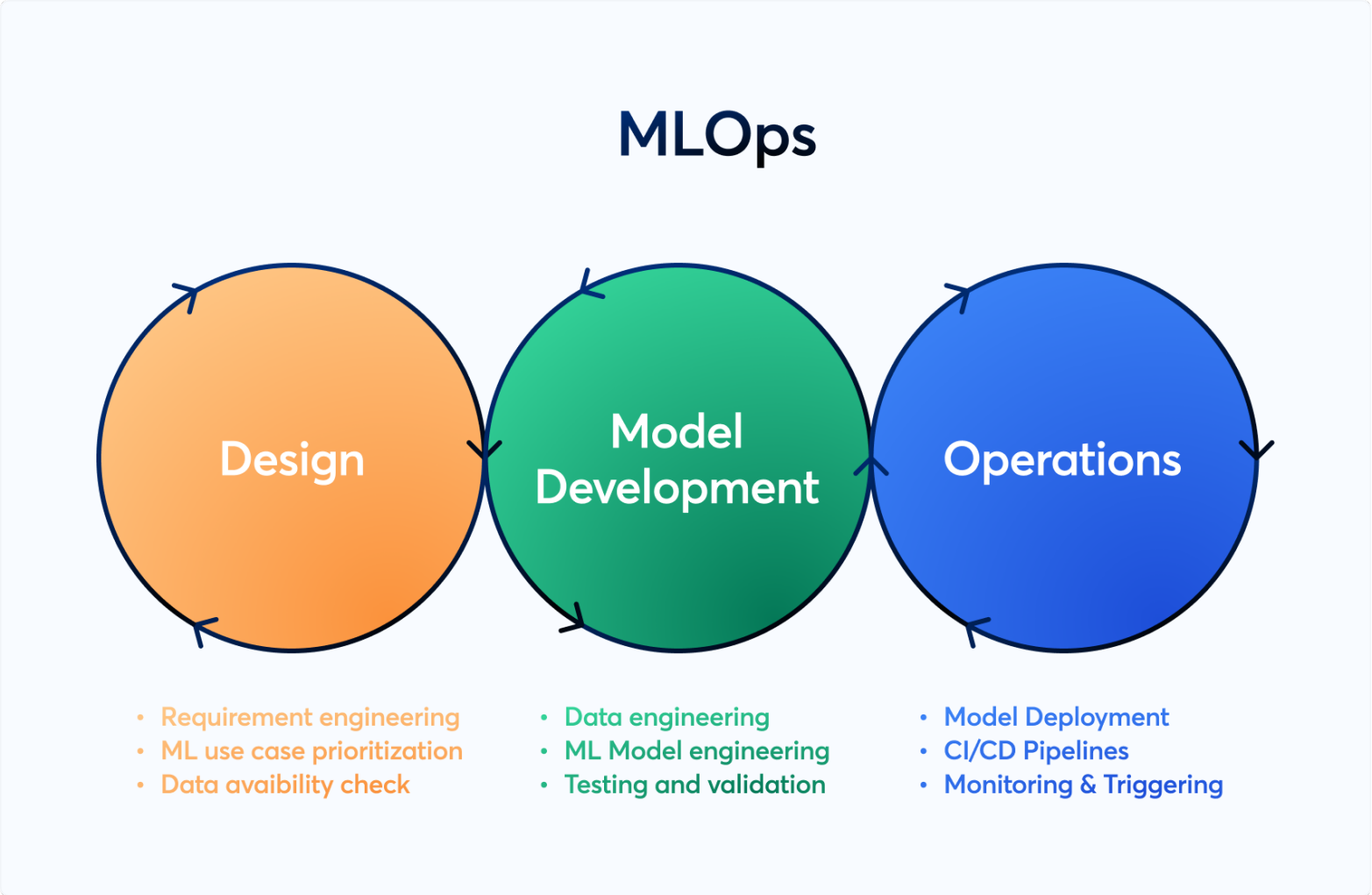
How to get most of the course?

- 3 examples: IMAGE GENERATOR, NEWS Briefer and PENGUIN SCANNER
- You can work with your own project (old or new)

Why do we need to know MLOps?

- Bridging the Gap Between Experimentation and Production
- Ensuring Model Reliability and Scalability
- Facilitating Collaboration Among Teams
- Automating the Machine Learning Lifecycle
- Improving Model Monitoring and Management

MLOps


MLOps
Design
Model Development
Operations
• Requirement engineering
• ML use case prioritization
• Data availability check
• Data engineering
• ML Model engineering
• Testing and validation
• Model Deployment
• CI/CD Pipelines
• Monitoring & Triggering

Serverless Machine Learning

Serverless Machine Learning (ML) refers to running machine learning models without managing infrastructure, using cloud-based services that automatically handle scaling, execution, and provisioning of resources. It allows data scientists and developers to deploy and run ML models on-demand without maintaining dedicated servers.

Application Programming Interface?

Definition

1. It is a set of defined rules that enable different applications to communicate with each other.
2. It acts as an intermediary layer that processes data transfers between systems.

Usability

APIs simplify software development and innovation by enabling applications to exchange data and functionality easily and securely

Example: 'ex_jokes.py'

Types regarding availability:

- **Public API** is open and available for use by any outside developer or business. These are also called open or external APIs
- **Partner API** is only available to specifically selected and authorized outside developers or API consumers. It facilitates business-to-business activities
- **Private API** is intended only for use within the enterprise to connect systems and data within the business
  - Enterprise Private API & Internal Private API (inside specific application)

Types regarding protocol

REST (Representational State Transfer)

- The most commonly used architectural style for web APIs.
- Stateless, lightweight and uses standard HTTP methods (GET, POST, PUT, and DELETE).
- Typically exchanges data in JSON, though XML is also supported.
- Example: Twitter API, OpenAI API

Types regarding protocol

SOAP (Simple Object Access Protocol)

- A protocol that follows strict messaging standards using XML.
- More secure and reliable, but heavyweight compared to REST.
- Commonly used in enterprise and financial systems where security and ACID compliance are critical (atomicity, consistency, isolation, and durability).
- Example: Paypal API

Types regarding protocol

RPC (Remote Procedure Call)

- One of the earliest API protocols, designed to execute functions on a remote server.
- Can be **XML-RPC** (uses XML for communication) or **JSON-RPC** (uses JSON).
- Typically used for lightweight and fast interactions but less flexible than REST.
- Example: Bitcoin's JSON-RPC interface

Types regarding protocol

WebSockets (Event-driven or Asynchronous APIs)

- Provides a **persistent** full-duplex communication channel between client and server.
- Unlike traditional request-response APIs, WebSockets push updates in real-time.
- Used in applications requiring **low-latency communication**, such as stock trading, chat apps, and multiplayer gaming.
- Example: WhatsApp

Types regarding protocol

GraphQL

- A query language for APIs, allowing clients to request exactly the data they need.
- Reduces over-fetching and under-fetching of data compared to REST.
- Used in modern applications that need **flexible** and **efficient** data retrieval.
- Example: GitHub GraphQL API

Types regarding protocol

gRPC (Google Remote Procedure Call)

- A high-performance RPC framework using **Protocol Buffers (protobuf)** instead of JSON/XML.
- Supports multiple programming languages and is commonly used in microservices architectures.
- Faster and more efficient than REST, particularly in inter-service communication.
- Example: Google Cloud APIs

What is an API Key?

- An **API key** is a unique identifier used to authenticate a request to an API.
- Typically included in the request header or as a query parameter.
- Provides **security** and helps to monitor and limit API usage.

Example: 'ex_finance.py', 'ex_news.py'

API Authentication Methods

- **API Key** (simple but less secure)
- **OAuth 2.0** (more secure, used for user authentication)
- **JWT (JSON Web Token)** (used for secure, token-based authentication)

What is Two-Factor Authentication (2FA)?

- 2FA adds an extra layer of security by requiring two different factors for authentication.
- Common second factors:
  - **SMS Codes** – A one-time password (OTP) sent via text message.
  - **Google Authenticator/Authy** – Time-based one-time passwords (TOTP).
  - **Biometric Authentication** – Fingerprint, Face ID, etc.
  - **Hardware Security Keys** – Such as YubiKey or Titan Security Key.

Examples of Applications Using 2FA

Authentication Method	Example Apps	2FA Support
API Key	OpenAI API	✗(Not user-based)
OAuth 2.0	Google, Binance	✓(e.g., SMS, TOTP)
JWT	Auth0, AWS Cognito	✓(e.g., TOTP, Hardware Keys)

Best Practices for API Keys

- Never hardcode API keys in source code.
- Use **environment variables** or secret managers.
- Limit access with API key restrictions.
- Rotate keys periodically for security.

Requests:

- GET - is a read-only operation and doesn't change the state of the resource but only retrieve data
- POST - sends data to a server to create or update a resource and is often used when submitting web forms
- PUT - is used to update an existing resource with new data
- PATCH - is used to apply partial modifications to a resource
- DELETE - used to delete a specified resource from the server

FastAPI

FastAPI is a modern, high-performance web framework for building APIs with Python 3.7+

- Designed for speed, nearly as fast as **Node.js and Go**.
- Built on **Starlette** and **Pydantic** for performance and data validation.
- Supports both **synchronous** and **asynchronous** code execution.
- **Minimal Code**: A basic API can be created in just 7-8 lines.
- **Automatic Documentation**: Provides built-in **Swagger UI** and **ReDoc** documentation.
- **Type Safety**: Uses Python's type hints for **data validation** and **autocomplete**.

What is an API Wrapper?

Definition

An API Wrapper is a set of programming instructions that acts as an intermediary layer between an application and the web API it intends to communicate with. It simplifies the API calls by providing more intuitive functions that are easier to work with.

- Simplifies the process of making API calls.
- Handles data formatting, error handling, and connection logic.
- Can provide additional functionality like caching responses.

Example: 'ex_datasets.py'

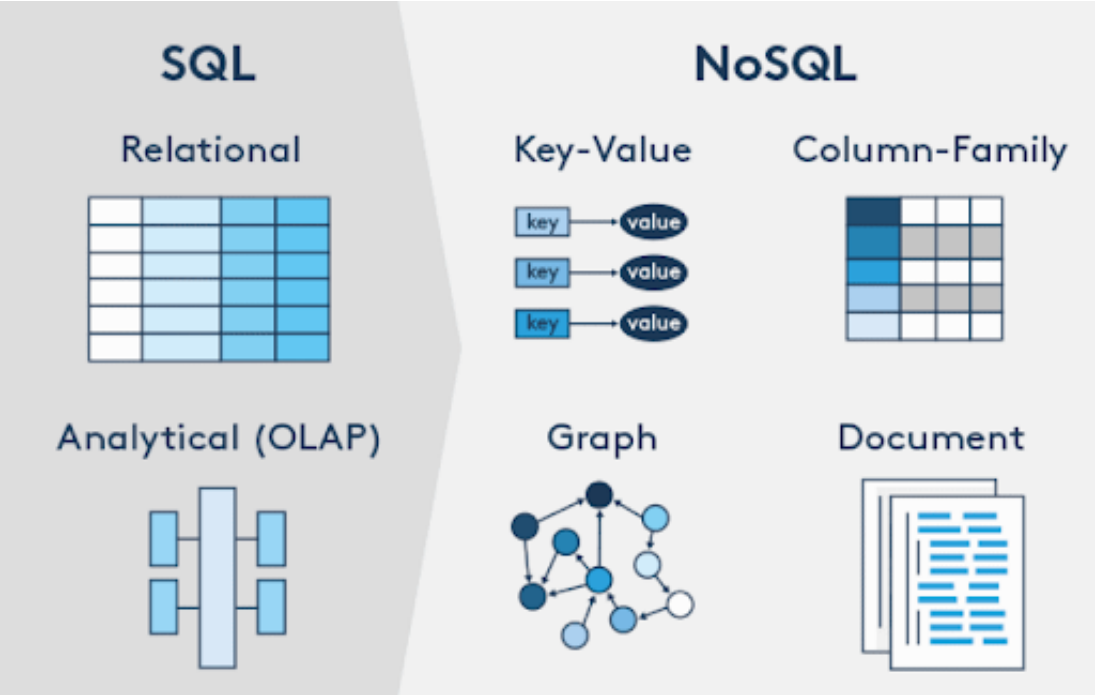
Why We Need Databases in MLOps

- Feature Store: Centralize feature storage and management for consistency across projects.
- Data Versioning: Track changes in datasets to ensure reproducibility.
- Experiment Tracking: Store experiment metadata and results for comparison.
- Model Management: Keep versions of models for deployment and rollback.

Types of Databases Used in MLOps

- Relational Databases (SQL): PostgreSQL, MySQL for structured data.
- NoSQL Databases: MongoDB, Cassandra for unstructured or semi-structured data.
- Time Series Databases: InfluxDB for time-dependent data.
- Graph Databases: Neo4j for complex, interconnected data relationships.
- Feature Stores: Specialized databases like Feast for managing and serving features to ML models.

Types of Databases

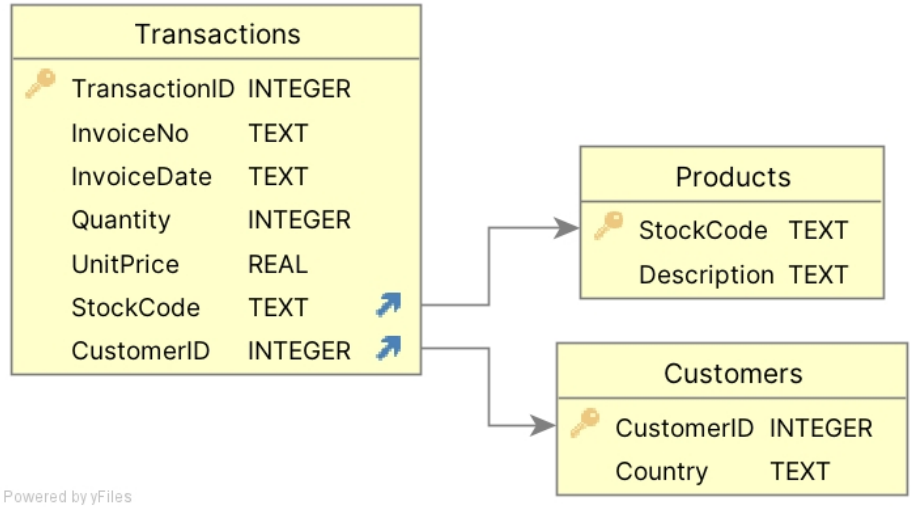


SQL Databases

SQL (Structured Query Language) Databases, also known as relational databases, are characterized by their structured format using tables.

- **ACID Properties:** Ensures database transactions are processed reliably (Atomicity, Consistency, Isolation, Durability).
- **Schema-based:** Requires a predefined schema for data storage, enhancing data integrity.
- **Relational Data Management:** Efficiently manages data across multiple tables through relationships.

SQL Databases



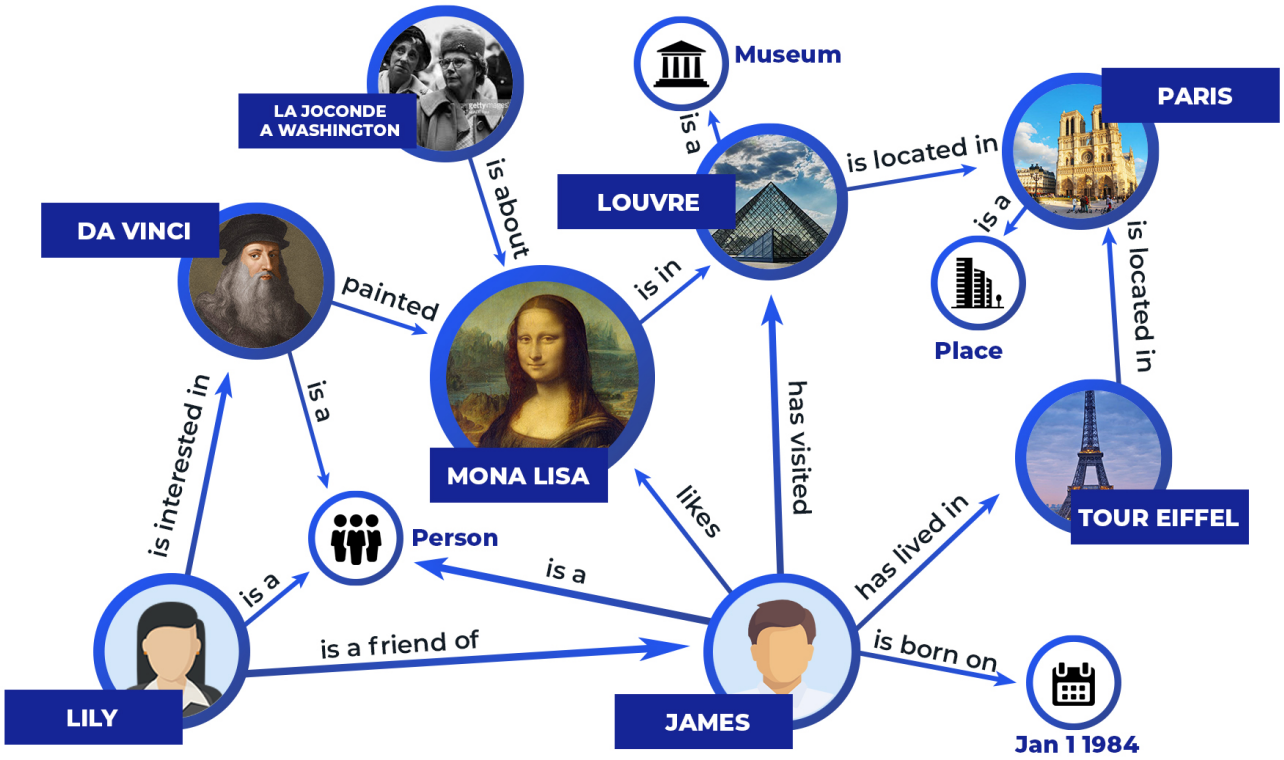
Example: 'db_create.py'

Knowledge Graph Databases

Knowledge Graph Databases are designed to store interlinked descriptions of entities — objects, events, or concepts — providing a framework that integrates data using a graph.

- **Graph Structure:** Data is represented as nodes (entities) and edges (relationships), enabling direct representation of relationships.
- **Semantic Querying:** Supports queries that understand the meaning of data, allowing for more nuanced data retrieval.
- **Schema-less or Schema-flexible:** Can evolve over time without requiring predefined schema modifications.

Knowledge Graph Database



How to get weather data from online API?

TASK: We want to create a simple web app showing recent and historical weather and air pollution data.

There are many websites but not all offer APIs, e.g., DMI.

We can get data for free on https://open-meteo.com/.

Exercise competition

You have 15 minutes to get data on Air Quality (PM 2.5) for Roskilde and Beijing

Hint: AirQuality API is in OtherAPIs :)